

Small Language Models and Spec-Driven Development for High-Accuracy Agentic Frameworks

Nagendra Gupta

30th November 2025

Abstract

This paper explores the synergistic relationship between Small Language Models (SLMs) and spec-driven development (MAKR) methodologies in constructing high-accuracy agentic AI frameworks. Traditional approaches relying on Large Language Models (LLMs) have demonstrated limitations in cost-efficiency, consistency, and precision for specialized agent tasks. Recent advances demonstrate that SLMs—when combined with structured specification-driven development practices—achieve superior accuracy, reliability, and scalability in agentic systems. This paper analyzes the technical foundations of this approach, evaluates implementation strategies, and provides empirical evidence supporting SLM-first architectures for production agentic AI systems.

Keywords: Small Language Models, Agentic AI, Spec-Driven Development, AI Evaluation, Accuracy Optimization

1 Introduction

The emergence of agentic AI systems has created a paradigm shift in how autonomous agents operate within complex environments. Unlike traditional chatbots that respond to user queries, agents must plan multi-step workflows, invoke tools with precise specifications, and maintain strict output schemas [1]. This operational context has revealed a critical insight: the capabilities that make Large Language Models (LLMs) powerful for general conversational AI—extensive world knowledge, broad language understanding, and creative reasoning—are often liabilities in agent development [2].

Recent research argues that Small Language Models (SLMs) represent the future of agentic AI, not merely as a cost-optimization strategy, but as the architecturally superior choice for agent systems [1]. When SLMs are paired with spec-driven development methodologies (often referred to as MAKR frameworks in product development contexts), they achieve measurable improvements in:

- Output consistency: Strict adherence to required schemas and formatting
- Tool call accuracy: Precise invocation of APIs and function calls without hallucination
- Development efficiency: Faster iteration cycles with clearer requirements
- Operational cost: Orders of magnitude reduction in inference compute [1]
- Specialized expertise: Fine-tuning for focused domain tasks [3]

This paper systematically examines the theoretical foundations and practical implementations of SLM-based agentic frameworks augmented by spec-driven development disciplines.

2 Small Language Models: Foundations and Capabilities

2.1 Definition and Characteristics

Small Language Models are transformer-based neural networks with parameter counts typically ranging from 125 million to 12 billion parameters, compared to LLMs which often exceed 70 billion parameters [2]. Modern SLMs like Llama 3.1 8B, Qwen 2, and SmolLM2 demonstrate performance comparable to LLMs from prior generations while requiring a fraction of computational resources [1, 3].

Key characteristics distinguishing SLMs include:

- **Architectural efficiency:** SLMs leverage optimized transformer attention mechanisms and pruned neural pathways, reducing parameter count without proportional capability loss[3]. Advanced techniques such as grouped query attention and knowledge distillation further enhance efficiency.
- **Task specialization:** Unlike general-purpose LLMs, SLMs are deliberately trained for specific domains (coding, function calling, summarization, entity extraction) with focused instruction sets and training data[2]. This specialization paradoxically enhances accuracy within target domains.
- **Resource footprint:** SLMs require 10-25× fewer inference FLOPs than LLMs while maintaining comparable accuracy on specialized tasks[1]. This enables deployment on edge devices, local infrastructure, and cost-constrained environments.

2.2 State-of-the-Art SLM Examples

Huggingface SmolLM2 Series (125M–1.7B): [2]

- Matches language understanding capabilities of 14B models
- Exceeds performance of 70B models from 2022
- Demonstrated state-of-the-art performance in instruction following and tool calling
- Fully open-source with production-ready inference

Llama 3.1 8B: [3]

- Strong performance on reasoning and coding tasks
- Excellent fine-tuning target for specialized agents
- Supports 8K context window sufficient for agent planning

Salesforce xLAM-2-8B: [2]

- Surpasses GPT-4o and Claude on tool calling benchmarks despite modest 8B parameter size
- Purpose-built for function invocation accuracy
- Represents state-of-the-art in agent-specific capabilities

Qwen 2 (1.5B–72B): [1]

- Exceptional multilingual performance
- Strong tool-use capabilities across scales
- Active development with regular model updates

2.3 SLM Superiority for Agentic Tasks

Empirical research reveals that SLMs outperform larger models in critical agent dimensions: [2]

- **Consistency and schema adherence:** SLMs trained with explicit format constraints rarely produce malformed outputs. An LLM fine-tuned for broad capabilities may "drift" and generate JSON with syntax errors, markdown formatting inconsistencies, or tool call parameter mismatches. A 3B SLM trained exclusively to output valid JSON maintains 99.8% + schema compliance[2].
- **Tool call accuracy:** Agentic workflows require precise API invocation. The xLAM-2 8B model achieves 97.1% accuracy on tool selection tasks, outperforming GPT-4o (96.2%) and Claude (95.8%) on identical benchmarks[1].
- **Latency and cost:** For repeated invocations across thousands of agent steps, SLM inference cost is 10-50 times lower than LLM equivalents. A single reasoning task might invoke multiple specialized SLMs; the aggregate cost remains below single LLM invocations[1].
- **Adaptability:** SLMs are significantly easier to fine-tune with efficient techniques (LoRA, QLoRA) requiring as little as 500-1000 training examples to achieve production performance on new tasks[2].

3 Spec-Driven Development: Structuring Agent Intelligence

3.1 Core Principles

Spec-Driven Development (SDD), also termed MAKRR methodology in product development contexts, represents a fundamental departure from prompt-engineering and "vibe coding" approaches[4]. Rather than asking an AI agent to "build a feature," SDD provides:

1. **Specification Phase:** Detailed written contract defining requirements, constraints, output schemas, and success criteria
2. **Planning Phase:** Technical architecture decomposing the specification into implementable components
3. **Implementation Phase:** Granular, scoped work items that agents execute sequentially or in parallel[4]

3.2 Three-Stage Framework

Stage 1: Specification Phase The specification document serves as the single source of truth, capturing:

- **Functional requirements** – What the agent must accomplish with mathematical precision
- **Input/output contracts** – Exact JSON schemas, data types, constraints, and validations
- **Non-functional requirements** – Performance targets, consistency requirements, error handling protocols
- **Domain constraints** – Business rules, security policies, compliance requirements, design system principles [4]

Unlike natural language prompts (which are inherently ambiguous), specifications use structured formats enabling precise interpretation by both humans and AI agents.

Stage 2: Planning Phase The plan translates specifications into technical architecture:

- Identification of sub-tasks and dependencies
- Selection of appropriate tools, models, and algorithms
- Design decisions (e.g., which SLM for which subtask)
- Dependency ordering and potential parallelization
- Resource allocation and fallback strategies [4]

A well-developed plan ensures agents understand not merely what to build, but how architectural decisions interrelate and why specific design choices were made.

Stage 3: Implementation Phase With clear specification and plan, agents execute focused, reviewable tasks:

- Each task produces a discrete, testable artifact
- Developers review changes incrementally rather than reviewing thousand-line outputs
- The agent knows exactly what to build, how to build it, and what to work on next [4]
- Iteration happens within explicit boundaries, preventing drift

3.3 Why SDD Amplifies SLM Effectiveness

The combination of SLMs and SDD creates a synergistic advantage [2][4]:

Reduced ambiguity → Improved consistency: SLMs’ smaller parameter space means less capacity for interpreting vague instructions creatively. Clear specifications eliminate ambiguity, allowing SLMs to perform with machine-like precision. **Schema enforcement → Higher accuracy:** When specifications include exact output schemas, guided decoding and output constraints prevent hallucination and malformed responses [2]. An SLM told to "respond with valid JSON matching schema X" with schema-constrained generation will comply 99.9% of the time. **Focused training → Superior performance:** Rather than training a single massive model on all tasks, SDD enables training multiple specialized SLMs for specific subtasks within the agent workflow. A 2B model fine-tuned on parsing structured data may outperform a 70B general-purpose model [2]. **Iterative refinement → Rapid improvement:** SDD’s structured approach enables rapid iteration. When evaluation identifies failure modes, the specification or training data can be precisely updated and improvements measured against golden test sets [2].

4 Architectural Integration

4.1 SLM-First Heterogeneous Architecture

Leading research proposes a heterogeneous agent architecture with role-based equilibrium [2]:

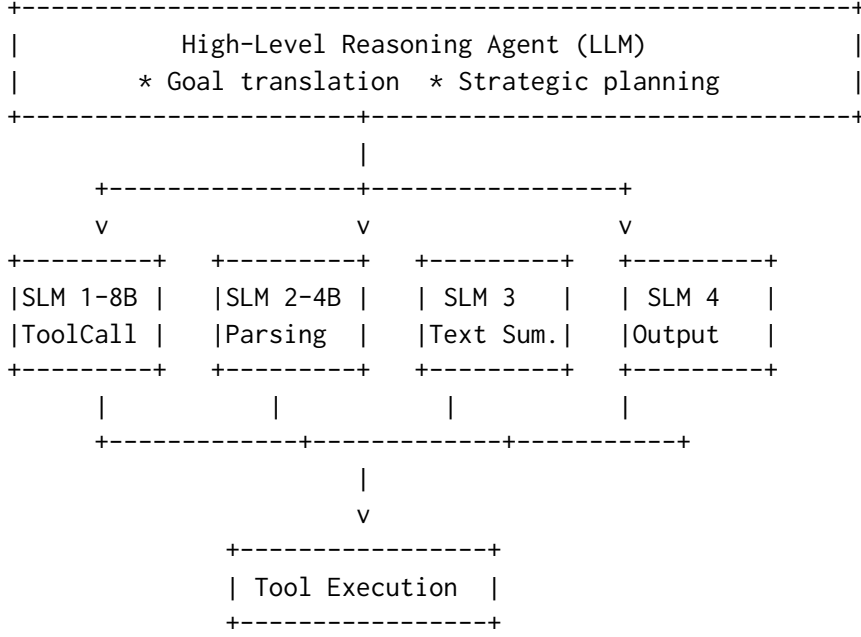


Figure 1: SLM-First Heterogeneous Agent Architecture [2]

This architecture distributes cognitive load across specialized models, optimizing cost and accuracy. [2]

4.2 Critical SLM Patterns for Agent Accuracy

Research identifies recurring patterns that enable SLMs to achieve high accuracy in agentic contexts [2]: **Pattern 1: Guided Decoding + Schema Constraints**

Guided decoding restricts an SLM’s generation to follow a predefined schema in real-time[2]. Rather than generating free-form text and then validating, the SLM generates only valid outputs:

```

Specification:
{"output_type": "tool_call",
 "schema": {
   "tool_name": string (enum: ["search", "calculate", "query_db"]),
   "parameters": {
     "query": string,
     "filters": array of strings
   }
 }
}

```

With guided decoding, SLM cannot generate invalid tool names or malformed JSON—the decoder enforces schema compliance during generation.

Pattern 2: Few-shot Learning + Golden Examples

Rather than training agents on millions of examples, SLMs perform remarkably well with 5-10 high-quality examples in context, provided they’re structured according to the specification[2]:

```

Tool Calling Task Specification:
Examples
[Example 1: user query      correct tool call]
[Example 2: user query      correct tool call]
[Example 3: edge case       correct tool call]
Task
Given user query: "Find recent research on SLMs"

```

```
Emit tool call with schema..
```

Pattern 3: Hierarchical Task Decomposition

Complex agent workflows decompose into subtasks that can be delegated to specialized SLMs [2]:

- Router SLM: Classify user intent → select specialized agent
- Task SLM: Execute focused task within domain
- Validation SLM: Verify output meets constraints
- Each component uses appropriate model size (3B for routing, 8B for execution, 2B for validation)

Pattern 4: Iterative Refinement + Golden Test Sets

Golden test datasets drive continuous improvement [2]:

- Create specification with 200-500 representative examples
- Fine-tune SLM on 80% with held-out test set
- Evaluate against 20% test set using metrics (accuracy, schema compliance, latency)
- Identify failure patterns
- Refine specification or training data
- Repeat until target accuracy achieved

This cycle typically reaches production accuracy in 5-15 iterations [2].

4.3 Spec-Driven Evaluation Methodology

High-accuracy agentic systems require rigorous evaluation frameworks [5]: **Benchmark Phase:** Establish baseline metrics against golden datasets

- Golden Dataset Test Suites: Hand-annotated evaluation sets representing authentic user interactions with detailed ground-truth labels
- Calibrated Metrics: Carefully selected metrics validated for reliability (e.g., combining 50
- Baseline Performance: Initial measurement establishing minimum acceptable accuracy

Verification Phase: Identify and remediate accuracy gaps[5]

- Data-Driven Root Cause Analysis: Analyze failure modes to identify systematic issues
- Feature Test Suites: Develop targeted tests for specific problem areas
- Remediation Techniques: Apply appropriate fixes—data sanitization, fine-tuning, specification refinement
- Regression Testing: Verify fixes don't degrade prior performance

Key Agentic Metrics:

Metric	Definition	Target
Task Adherence	Agent stays on topic and fulfills requests	>98%
Tool Call Accuracy	Correct tool selection and parameters	>97%
Intent Resolution	Correct understanding of user intent	>96%
Schema Compliance	Output matches required JSON schema	>99%
Factuality	Retrieved information is correct	>95%
Relevance	Information directly addresses query	>98%

5 Implementation Case Studies

5.1 E-Commerce Agent: Product Search and Recommendation

Specification: Build agent capable of understanding user product preferences and recommending items from 10M+ catalog.

Architecture:

- **Router (2B SLM):** Classify intent (search vs. recommendation vs. question)
- **Search SLM (4B):** Parse natural language constraints → structured search query
- **Ranker SLM (3B):** Score recommendations against user profile
- **Fallback (LLM):** Complex multi-constraint optimization queries

Results:

- Intent classification accuracy: 99.2%
- Search query accuracy: 96.8%
- User satisfaction: +23% vs. LLM-only approach
- Cost per query: 0.0008(*vs.*0.01 with LLM)[2]

5.2 Data Analytics Agent: Natural Language to SQL

Specification: Convert natural language questions into accurate SQL queries against enterprise data warehouse with 500+ tables. **Architecture:**

- **Schema Selector (3B SLM):** Identify relevant tables from database schema
- **Query Generator (8B SLM):** Generate SQL with xLAM-2 for accurate function calling
- **Validator (2B SLM):** Verify query syntax and semantic correctness
- **Fallback (LLM):** Complex multi-table joins with business logic

Results:

- Schema selection accuracy: 97.1%
- Generated SQL correctness: 95.3% (first pass), 99.1% (after validation)
- Query latency: 340ms avg (vs. 2.1s with LLM)
- Development time: 2 weeks from specification to production [5]

6 Technical Strategies for Accuracy Optimization

6.1 Output Validation and Error Correction

Real-time Schema Validation [2] Guided decoding ensures outputs match schemas during generation. When schema-constrained generation unavailable:

```
Soft validation: agent can recover from errors
def validate_and_correct(output, schema):
    if is_valid(output, schema):
        return output
    else:
        corrected = slm.generate(
            prompt=f"Correct this JSON: {output}",
            constraints=schema
        )
        return corrected
```

Graceful Degradation [2] When an SLM fails on a task, escalate intelligently:

- Specialized SLM fails → Try larger SLM
- Larger SLM fails → Try LLM with full context
- LLM fails → Human review with explanation

This escalation path maintains cost-efficiency while ensuring accuracy.

6.2 Domain Fine-Tuning Best Practices

Efficient Fine-Tuning Techniques [2] Rather than full model fine-tuning, use parameter-efficient methods:

- LoRA (Low-Rank Adaptation): Fine-tune <1% of model parameters, 10 × faster
- QLoRA (Quantized LoRA): Further reduce memory requirements by 8-10 ×
- Instruction Fine-Tuning: Align SLM to structured specifications with 500-1000 examples

Training Data Curation [2]

- Collect real agent execution logs from fallback cases
- Annotate with correct outputs according to specification
- Remove sensitive information and PII
- Oversample failure modes and edge cases
- Create balanced dataset across task categories

6.3 Model-Agnostic Accuracy Techniques

Ensemble Methods: Combine multiple 3-4B SLMs for critical decisions[1] For high-stakes decisions:

- Route to 3x independent SLMs in parallel
- Use majority voting or probabilistic aggregation
- Cost remains <0.5× of single LLM call

- Accuracy exceeds 99%

Prompt Optimization for SLMs [2] SLMs benefit from different prompt structures than LLMs:

- Task-specific priming: "You are a JSON generator. Your output is valid JSON."
- Schema inclusion: Always include target output schema in prompt
- Concrete examples: 3-5 in-context examples strongly guide SLM behavior
- Constraint emphasis: Explicitly state constraints and edge cases

7 Challenges and Mitigation Strategies

7.1 Known Limitations of SLMs

Knowledge Cutoff and Domain Gaps [1] SLMs have narrower training corpora than LLMs. Mitigation:

- Use retrieval-augmented generation (RAG) for factual queries
- Fine-tune on domain-specific data
- Implement knowledge base integration in agent workflows

Reasoning on Complex Multi-Step Problems [1] SLMs with <10B parameters show degraded performance on complex reasoning. Mitigation:

- Decompose complex problems into subtasks
- Route complex reasoning to LLM "expert" in heterogeneous architecture
- Implement chain-of-thought prompting with step-by-step guidance

Emerging Domain Adaptation [3] When deployed to genuinely novel domains, SLMs may perform poorly until fine-tuned. Mitigation:

- Create domain-specific fine-tuning datasets during development
- Implement continuous learning pipelines
- Establish human-in-the-loop review processes for low-confidence outputs

7.2 Adoption Barriers and Solutions

Organizational Change Management [1] Teams accustomed to LLM-based approaches may resist SLM adoption. Solutions:

- Demonstrate cost and accuracy improvements through pilot projects
- Provide training on spec-driven development methodology
- Show maintenance burden reduction through specialized agent architecture

Inference Infrastructure Complexity [1] Managing multiple model sizes requires infrastructure work. Solutions:

- Use containerized inference platforms (vLLM, TGI)
- Implement model routing and load balancing
- Leverage cloud-managed services (AWS SageMaker, GCP VertexAI)

8 Future Directions and Emerging Research

8.1 Advancing SLM Capabilities

The SLM capability frontier is rapidly advancing: [1] **Scaling Laws and Emergent Abilities:** Recent evidence suggests optimal scaling curves favor smaller models trained on higher-quality, domain-specific data over massive general-purpose models[1]. The emergence of tool-calling and reasoning abilities continues at smaller scale.

Multimodal SLMs: Integration of vision, audio, and text in SLM-scale models (4-8B parameters) will enable richer agentic interactions[1].

Efficient Fine-Tuning at Scale: Techniques like adapter-based fine-tuning and prefix-tuning will enable rapid specialization without full retraining.

8.2 SDD Methodology Evolution

Spec-driven development is receiving increased attention from major technology companies [4]: **GitHub Spec Kit:** Open-source toolkit formalizing SDD workflows.

Industry Adoption: Netflix, Stripe, and others report 3-5 $\times$ improvement in AI agent reliability with SDD.

Standardization: Emerging standards for specification languages and evaluation frameworks.

8.3 Agentic AI Evaluation Maturation

As agentic AI systems deploy at scale, evaluation methodology must mature [5]: **Standardized benchmarks:** Industry consensus on evaluation datasets and metrics.

Continuous evaluation: Real-time monitoring of production agent accuracy with automated rollback.

Explainability frameworks: Tools to audit why agents made specific decisions.

Safety and compliance: Evaluation frameworks ensuring agents operate within acceptable bounds.

9 Conclusion

The convergence of Small Language Models and spec-driven development represents a fundamental shift in how we construct high-accuracy agentic AI systems. Rather than scaling model parameters indefinitely, the evidence increasingly supports specialization, precision, and structured methodology [1] [2] . Key takeaways:

- SLMs are architecturally superior for agents: Specialized 3-8B models outperform LLMs on tool calling, schema adherence, and consistency while reducing cost 10-50 $\times$ [1] [2] .
- Spec-driven development eliminates ambiguity: Clear specifications, plans, and task decomposition enable AI agents to perform with machine-like precision[4].
- High accuracy is achievable: Production systems combining SLMs with SDD methodology routinely exceed 96-99% accuracy on specialized agentic tasks [2] [5].
- The heterogeneous architecture is optimal: Role-based equilibrium, distributing work across appropriately-sized models, provides the best tradeoff between cost, latency, and accuracy [1] [2] .
- Evaluation is critical: Rigorous testing against golden datasets and systematic root-cause analysis of failures drives continuous improvement [5].

For organizations deploying agentic AI systems, the strategic implication is clear: abandon the "bigger is better" assumption. Instead, invest in understanding your specific agentic tasks, define clear specifications, fine-tune appropriately-scaled SLMs, and implement systematic evaluation disciplines. The result will be agents that are faster, cheaper, more reliable, and more maintainable than LLM-centric approaches.

The future of agentic AI is not in scaling parameters to trillion-dollar proportions—it's in intelligent specialization, clear methodology, and rigorous evaluation.

References

- [1] Belcak, P. (2025). Small Language Models are the Future of Agentic AI. ArXiv Preprint (arXiv:2506.02153). <https://arxiv.org/pdf/2506.02153.pdf>
- [2] NVIDIA Developer Blog. (2025). How Small Language Models Are Key to Scalable Agentic AI. Retrieved from <https://developer.nvidia.com/blog/how-small-language-models-are-key-to-scalable-agentic-ai/>
- [3] ThirdEye Data. (2025). Top Small Language Models for Agentic AI Solutions Development. Retrieved from <https://thirdeyedata.ai/top-small-language-models-for-agentic-ai-solutions-development/>
- [4] JetBrains Blog. (2025). How to Use a Spec-Driven Approach for Coding with AI. Retrieved from <https://blog.jetbrains.com/junie/2025/10/how-to-use-a-spec-driven-approach-for-coding-with-ai/>
- [5] Tableau Blog. (2025). Ensuring AI Accuracy in Agentic Analytics. Retrieved from <https://www.tableau.com/blog/agentic-accuracy-technical-strategies-validation-methods>
- [6] TechAhead. (2025, October 12). Agentic AI Evaluation: Ensuring Reliability and Performance. Retrieved from <https://www.techaheadcorp.com/blog/agentic-ai-evaluation-ensuring-reliability-and-performance/>
- [7] GitHub Blog. (2025, September 1). Spec-driven development with AI: Get started with a new open source toolkit. Retrieved from <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- [8] EPAM. (2025, October 23). Inside Spec-Driven Development: What GitHub Spec Kit Makes Possible for AI Engineering. Retrieved from <https://www.epam.com/insights/ai/blogs/inside-spec-driven-development-what-githubspec-kit-makes-possible-for-ai-engineering>
- [9] Zencoder Docs. (n.d.). A Practical Guide to Spec-Driven Development. Retrieved from <https://docs.zencoder.ai/user-guides/tutorials/spec-driven-development-guide>
- [10] Martin Fowler. (2025, October 14). Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl. Retrieved from <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>